RV32IM CPU in Verilog

Design Notes: 5-Stage Pipeline, Part 1

Jonathan Avalos

October 12, 2026

Contents

1 Overview **2**

2 The five stages **2**

3 Pipeline registers **5**
3.1 IF/ID (96 bits) . . . 5
3.2 ID/EX (183 bits) . . . 5
3.3 EX/MEM (141 bits) . . . 6
3.4 MEM/WB (136 bits) . . . 6

4 core_pipe **6**
4.1 IF . . . 7
4.2 ID . . . 7
4.3 EX . . . 8
4.4 MEM and WB . . . 9

5 Hazards (not handled yet) **9**
5.1 Data hazards: 3 instructions between a write and a read . . . 9
5.2 Control hazards: 2 instructions after a branch or jump . . . 10

6 Verification **10**

7 Next steps **10**

1 Overview

This document picks up where the Stage 1 and Stage 2 notes left off. Since then the single-cycle RV32IM core has been split into a 5-stage pipeline, `core_pipe.v`. This first part of the pipeline has the stages and the pipeline registers, but **no hazard handling**: no forwarding, no stalls and no flushes. The single-cycle `core.v` is unchanged and still passes everything.

Status

- **Stage 1 and Stage 2:** single-cycle RV32IM core. Done (see the previous notes).
- **Pipeline, part 1:** 5 stages, 4 pipeline registers, branches resolved in EX. Done. Runs any program that leaves enough space between dependent instructions.
- **Verified** with a new hand-scheduled test program; all 10 results pass (Section 6).
- **Next:** hazard handling (forwarding, a load-use stall, a branch flush), then Stage 3 (CSRs, traps, timer, MMIO).

New files

file	what it is
`core_pipe.v`	the 5-stage core, same ports as `core.v`
`top_pipe.v`	`top.v` with `core_pipe` in place of `core`
`testbenches/test_asm_nohazards.S`	test program with the gaps filled by hand
`testbenches/top_pipe_tb.v`	testbench for the pipeline

Module hierarchy

No new modules were needed. `core_pipe` uses the same building blocks as `core`; only the wiring between them changed.

```
top_pipe
  core_pipe
    reg32           read in ID, written in WB
    immgen          ID
    decoder         ID
    alu             EX
    branch_unit     EX
    multdiv         EX
  memory            instruction port in IF, data port in MEM
```

2 The five stages

Each instruction now takes 5 clock cycles to finish, but a new one starts every cycle, so up to 5 instructions are in the CPU at once, each in a different stage.

stage	job	parts
IF	fetch the instruction at `pc`	`pc`, memory instruction port, the `pc + 4` adder
ID	work out what it is, read registers	`decoder`, `immgen`, `reg32` *reads*
EX	do the math, decide branches	mux A, mux B, `alu`, `multdiv`, `branch_unit`, the `pc + imm` adder, the next-PC mux
MEM	load or store	memory data port
WB	write the result to `rd`	the `wb_sel` mux, `reg32` *write*

- `reg32` belongs to two stages: it's read in ID and written in WB.
- The next-PC mux is built in EX, because that's where the branch decision and the targets are. Its output feeds back to the `pc` register in IF.
- The memory has two ports into one array (a modified Harvard layout): IF uses the instruction port and MEM uses the data port, so a fetch and a load can happen in the same cycle.

Figure 1 on the next page shows the single-cycle datapath cut into the five stages.

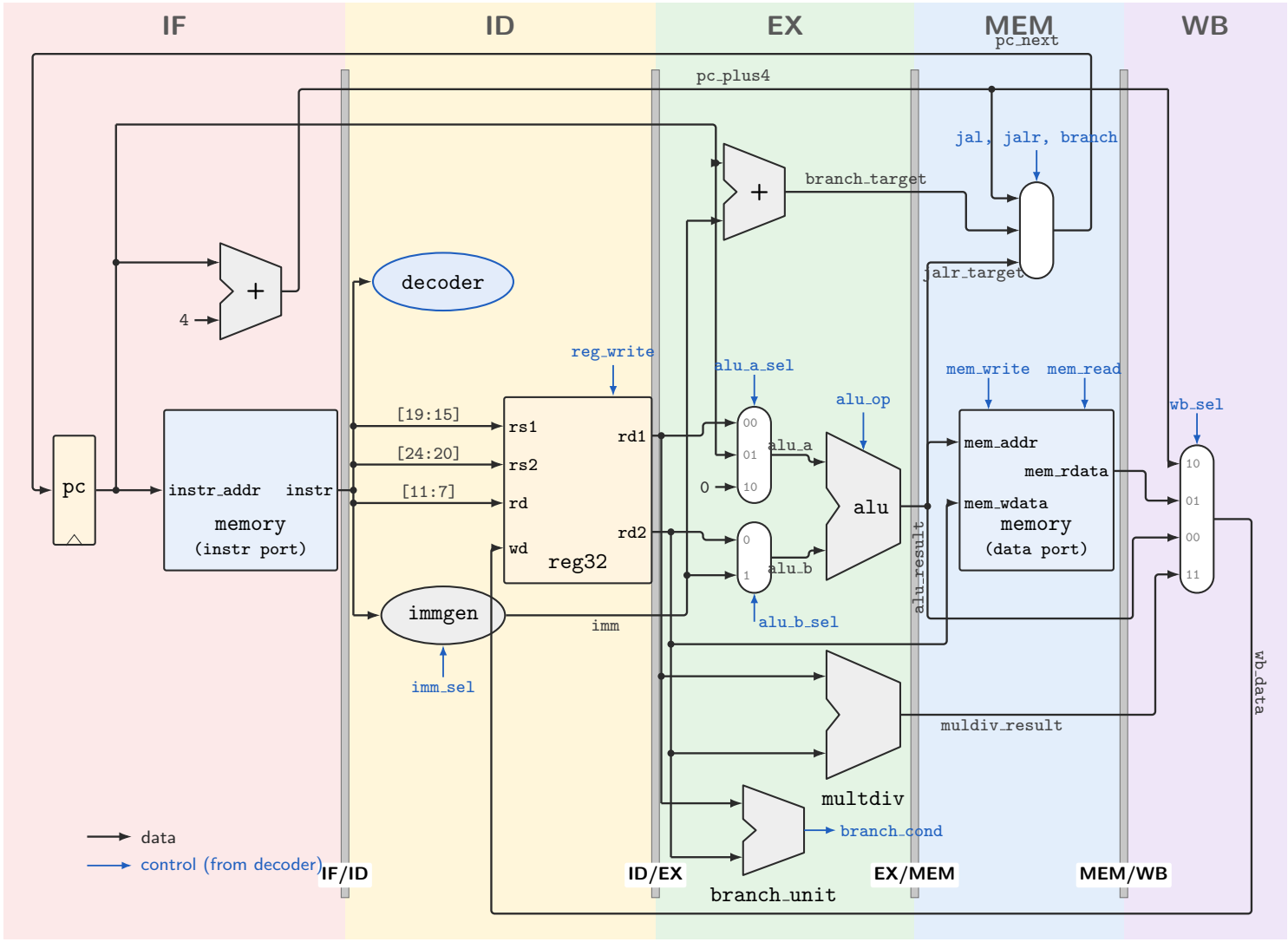


Figure 1: The single-cycle datapath split into the five stages. The gray bars are the pipeline registers. Every wire that crosses a bar from left to right has to be stored in that register; wires going right to left (`pc_next`, `wb_data`) are feedback and are not stored.

3 Pipeline registers

A signal has to be stored in every pipeline register between the stage that *makes* it and the last stage that *uses* it. The four registers below came out of the P1 worksheet.

Naming

Every wire has a prefix that says which stage it belongs to, and every pipeline register has a prefix that says which two stages it sits between:

prefix	meaning	example
`if_`, `id_`, `ex_`	a wire made in that stage, this cycle	`if_pc_plus4`
`if_id_`, `id_ex_`, `ex_mem_`, `mem_wb_`	a register between two stages	`if_id_pc_plus4`

This matters because the same name can hold different instructions' values in the same cycle. `id_rd` is the destination of the instruction being decoded; `mem_wb_rd` is the destination of the instruction three places ahead of it.

3.1 IF/ID (96 bits)

register	width	why it's kept
`if_id_instr`	32	decoded in ID
`if_id_pc`	32	EX needs it for `auipc` and the branch target
`if_id_pc_plus4`	32	WB writes it for `jal` / `jalr`

3.2 ID/EX (183 bits)

register	width	why it's kept
data		
`id_ex_pc`	32	mux A (`auipc`), `pc + imm`
`id_ex_pc_plus4`	32	passed on toward WB
`id_ex_rs1_data`	32	mux A, `multdiv`, `branch_unit`
`id_ex_rs2_data`	32	mux B, `multdiv`, `branch_unit`, store data
`id_ex_imm`	32	mux B, `pc + imm`
`id_ex_rd`	5	passed on toward WB
`id_ex_funct3`	3	`multdiv`, `branch_unit`, memory size
control		
`id_ex_alu_a_sel`, `id_ex_alu_b_sel`	2 + 1	the ALU muxes
`id_ex_alu_op`	4	`alu`
`id_ex_branch`, `_jal`, `_jalr`	3	next-PC mux
`id_ex_mem_read`, `_mem_write`	2	passed on to MEM
`id_ex_reg_write`, `id_ex_wb_sel`	1 + 2	passed on to WB

3.3 EX/MEM (141 bits)

register	width	why it's kept
`ex_mem_alu_result`	32	memory address, or the result for WB
`ex_mem_muldiv_result`	32	result for WB
`ex_mem_rs2_data`	32	store data
`ex_mem_pc_plus4`	32	passed on toward WB
`ex_mem_rd`	5	passed on toward WB
`ex_mem_funct3`	3	memory size and sign
`ex_mem_mem_read`, `_mem_write`	2	memory
`ex_mem_reg_write`, `ex_mem_wb_sel`	1 + 2	passed on to WB

3.4 MEM/WB (136 bits)

register	width	why it's kept
`mem_wb_mem_rdata`	32	loaded value
`mem_wb_alu_result`	32	ALU result
`mem_wb_muldiv_result`	32	multiply/divide result
`mem_wb_pc_plus4`	32	return address for `jal` / `jalr`
`mem_wb_rd`	5	which register to write
`mem_wb_reg_write`	1	whether to write
`mem_wb_wb_sel`	2	which result to write

That's 556 flip-flops in total.

Never stored

- `imm_sel`: made and used in ID (decoder to `immgen`).
- `rs1`, `rs2`: used to read `reg32` in ID and not needed after. They'll be added to ID/EX for forwarding, which has to compare them against later `rd`s.
- `branch_target`, `jalr_target`, `branch_cond`: made in EX and fed straight back to the PC.
- `instr`: once it's decoded, every piece that's still needed has its own register.

4 core_pipe

File: `core_pipe.v` **Purpose:** the pipelined datapath. It has the same ports and `RESET_PC` parameter as `core.v`, so `top_pipe.v` only has to change the module name.

The file is laid out in the order an instruction travels: the pipeline register declarations, then IF, ID, EX, MEM and WB. Each stage ends with the `always` block that loads the register after it.

4.1 IF

```
assign instr_addr  = pc;
assign if_pc_plus4 = pc + 4;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        pc             <= RESET_PC;
        if_id_pc       <= 0;
        if_id_instr    <= 32'h00000013; // addi x0, x0, 0
        if_id_pc_plus4 <= 0;
    end else begin
        pc             <= pc_next;
        if_id_pc       <= pc;
        if_id_instr    <= instr;
        if_id_pc_plus4 <= if_pc_plus4;
    end
end
```

- `if_pc_plus4` is IF's own `pc + 4`, a wire. It's a different thing from `if_id_pc_plus4`, the register that saves it at the clock edge. Mixing these two up was the first bug in this stage.
- `if_id_pc <= pc` saves the PC of the instruction being fetched, not `pc_next`. That's the address `auipc` and the branch adder need later.
- On reset, `if_id_instr` is loaded with `00000013`, which is `addi x0, x0, 0`, a `nop`. Otherwise ID would decode whatever was in the register at power-up.
- `pc_next` is declared here but driven in EX, where the branch decision is made.

4.2 ID

```
assign id_rs1    = if_id_instr[19:15];
assign id_rs2    = if_id_instr[24:20];
assign id_rd     = if_id_instr[11:7];
assign id_funct3 = if_id_instr[14:12];

reg32 u_reg32 (
    .clk(clk),
    .we(mem_wb_reg_write), // write comes from wb stage
    .reset(rst),
    .rs1(id_rs1),
    .rs2(id_rs2),
    .rd(mem_wb_rd),
    .wd(wb_data),
    .rd1(id_rs1_data),
    .rd2(id_rs2_data)
);
```

- The decoder and `immgen` are the same instances as in `core.v`, but they read `if_id_instr` and drive `id_` wires.

- **reg32 is split across two stages.** The read ports (`rs1`, `rs2`, `rd1`, `rd2`) belong to the instruction in ID. The write port (`we`, `rd`, `wd`) belongs to the instruction in WB, three instructions ahead. If the write port used `id_reg_write` and `id_rd`, every result would go into the wrong register at the wrong time.
- `id_rs1` (5 bits, a register number) and `id_rs1_data` (32 bits, its value) are different signals. ID/EX stores the data. Storing the number by mistake compiles without an error: the 5-bit value is just zero-padded.

The ID/EX `always` block copies every `id_` wire into its `id_ex_` register, and on reset clears all of them, including every control bit, so nothing writes or branches before the first real instruction arrives.

4.3 EX

```
assign ex_alu_a = (id_ex_alu_a_sel == 2'b01) ? id_ex_pc
                : (id_ex_alu_a_sel == 2'b10) ? 32'b0
                : id_ex_rs1_data;
assign ex_alu_b = (id_ex_alu_b_sel == 1'b1) ? id_ex_imm : id_ex_rs2_data;

assign ex_branch_target = id_ex_pc + id_ex_imm;
assign ex_jalr_target   = {ex_alu_result[31:1], 1'b0};

assign pc_next = (id_ex_jalr)                       ? ex_jalr_target
               : (id_ex_jal)                        ? ex_branch_target
               : (id_ex_branch && ex_branch_taken)  ? ex_branch_target
               : if_pc_plus4;
```

- The rule in EX: everything it *reads* comes from `id_ex_`, everything it *makes* is `ex_`.
- The muxes, the jump targets and the next-PC chain are the same logic as `core.v`, just with pipelined inputs.
- `ex_branch_target` uses `id_ex_pc`, not `pc`. By the time a branch is in EX, `pc` is already 2 instructions further along.
- `jal` has no target of its own: it jumps to `pc + imm`, the same sum as a branch. `immgen` already handled the different immediate layout back in ID.
- The fall-through case is `if_pc_plus4`, IF's `pc + 4`, because that's the instruction to fetch next if nothing jumps.
- `alu`, `multdiv` and `branch_unit` are the same instances as before, wired to `id_ex_` inputs.

The EX/MEM block stores the two results, the store data (`id_ex_rs2_data`), `pc_plus4`, `rd`, `funct3` and the four control bits that are still needed. `branch`, `jal` and `jalr` stop here.

4.4 MEM and WB

```
assign mem_addr   = ex_mem_alu_result;
assign mem_wdata  = ex_mem_rs2_data;
assign mem_write  = ex_mem_mem_write;
assign mem_read   = ex_mem_mem_read;
assign mem_funct3 = ex_mem_funct3;

assign wb_data = (mem_wb_wb_sel == 2'b00) ? mem_wb_alu_result
               : (mem_wb_wb_sel == 2'b01) ? mem_wb_mem_rdata
               : (mem_wb_wb_sel == 2'b10) ? mem_wb_pc_plus4
               : mem_wb_muldiv_result;
```

- MEM is only wiring: the memory ports are driven from EX/MEM.
- The MEM/WB block saves `mem_rdata` together with everything else WB needs.
- WB is a single `assign`: the same 4-to-1 mux as `core.v`. `wb_data` is declared up in ID, because `reg32` uses it there.

Notes

- The pipeline registers reset *asynchronously* (`posedge clk or posedge rst`). `core.v`'s PC and `reg32` reset synchronously. Both work in simulation; it's worth picking one style before synthesis.
- `` `default_nettype none `` is on, so a misspelled wire is an error instead of a silent 1-bit net. It caught most of the naming mistakes.

5 Hazards (not handled yet)

With no hazard hardware, a program only works if it spaces instructions out itself. There are two rules.

5.1 Data hazards: 3 instructions between a write and a read

A result is written into `reg32` at the clock edge at the *end* of WB. A later instruction reads its registers in ID. `reg32` has no write-through, so the read has to come *after* that edge.

cycle	1	2	3	4	5	6
`addi x1, x0, 5`	IF	ID	EX	MEM	**WB**	
`nop`		IF	ID	EX	MEM	WB
`nop`			IF	ID	EX	MEM
`nop`				IF	ID	EX
reads `x1`					IF	**ID**

With only 2 in between, the read would be in cycle 5 and get the old value. The 3 don't have to be `nop`s; any instruction that doesn't use the register works. With write-through in `reg32`, 2 would be enough.

5.2 Control hazards: 2 instructions after a branch or jump

Branches and jumps decide in EX. By then the next 2 instructions are already in IF/ID and ID/EX, and with no flush they run whether the branch is taken or not.

cycle	1	2	3	4
`beq`	IF	ID	**EX**	MEM
next		IF	ID	EX
next + 1			IF	ID
target				IF

So every branch and jump is followed by 2 `nop`s. This includes the final `j end`: without them the pipeline fetches the words after it.

6 Verification

The pipeline was tested with a new program, `test_asm_nohazards.S`, written by hand to follow the two spacing rules above. It loops over a 6-word array (sum, signed and unsigned max, min, negative count, product), divides, and calls a gcd function, storing 10 answers at `0x400`. The full write-up, including the program, the testbench and the register trace, is in the separate report, *Functional Verification of a 5-Stage RV32IM Pipeline Without Hazard Handling.*

Results

- All 10 answers and the final PC check pass on `core_pipe.v`.
- The same program gives the same answers on the single-cycle `core.v`.
- The pipeline takes 471 cycles against 366 for the single-cycle core. The extra 105 are the 51 taken branches and jumps at 2 cycles each (the 2 `nop`s after them run on the pipeline but are skipped on the single-cycle core), plus 3 cycles for the final store to reach MEM.

Building and running

```
cd testbenches && ./build_hex.sh test_asm_nohazards.S && cd ..
iverilog -o sim testbenches/top_pipe_tb.v top_pipe.v core_pipe.v memory.v \
    decoder.v immgen.v 32register.v register.v alu.v addersub.v logics.v \
    shifter.v compare.v multdiv.v branch_unit.v && vvp sim
```

Add `-DWATCH` to `iverilog` to print the registers as they change.

7 Next steps

1. Run the original `test_asm.S` on `top_pipe`. It has no spacing, so boxes should fail. Match each failure to a data or control hazard.
2. **Forwarding:** add `rs1` and `rs2` to ID/EX, and send EX/MEM and MEM/WB results straight back to the ALU inputs when their `rd` matches.

3. **Load-use stall:** a `lw` followed right away by an instruction that uses it still can't be forwarded in time, so hold IF and ID for one cycle.
4. **Flush:** on a taken branch or jump, turn the 2 instructions behind it into `nop`s.
5. Once those work, `test_asm.S` should pass on the pipeline unchanged, and the `nop`s can come out of `test_asm_nohazards.S`.
6. Then Stage 3: CSRs, traps, timer, MMIO.